A PRACTICAL FIELD GUIDE

Structured prompts and repeatable workflows for developers, computer science students, and technical learners.

CODE ? DEBUG ? LEARN ? BUILD

A practical toolkit for developers, computer science students, and technical learners who want more useful, repeatable results from AI.

Use the frameworks and templates in this guide with tools such as ChatGPT, Claude, or Gemini. Adapt the prompts to your language, codebase, course, and AI tool. AI can make mistakes: verify generated code, citations, explanations, and complexity claims before relying on them.

What this guide helps you do

- Write clearer technical prompts with the R-C-T-O framework.
- Refactor code while preserving behavior and considering complexity.
- Debug errors with relevant context instead of guesswork.
- Generate and review useful documentation.
- Study algorithms and data structures using the Feynman technique.
- Summarize technical papers and lecture notes.
- Plan features, practice active recall, and write consistent Git commits.

Who this is for

- **Computer science students** learning coursework, algorithms, and data structures.
- **Junior developers and programmers** improving code quality and debugging.
- **Tech creators and lifelong learners** using AI as a productivity aid for technical work.

Contents

1. Prompting for technical work
2. The R-C-T-O framework
3. Development and coding workflows
4. Study and research workflows
5. Productivity and exam preparation
6. Quick reference
7. Responsible use

1. Prompting for technical work

An AI response depends on the information and constraints in the prompt. A request such as “fix my code” leaves out the intended behavior, relevant environment, error details, and what a successful fix should look like.

Before asking, gather the context that matters:

- **Goal:** What should the program or explanation accomplish?
- **Context:** What language, framework, version, assignment, or code is relevant?

- **Evidence:** What input, output, error message, or observed behavior can you provide?
- **Constraints:** What must not change? Are dependencies, runtime, or style restricted?
- **Deliverable:** Do you want a patch, explanation, tests, a plan, or a comparison?

Share only code and information you are permitted to disclose. Remove credentials, tokens, personal information, and proprietary material you do not have permission to share.

2. The R-C-T-O framework

R-C-T-O is a practical prompt blueprint:

- **Role:** Define the perspective or expertise that would be useful.
- **Context:** Provide the task background, code, environment, and relevant facts.
- **Task:** State the specific work to perform.
- **Output:** Specify the format, level of detail, and any checks or constraints.

The framework helps reduce ambiguity; it does not guarantee correctness or prevent hallucinations. Ask the AI to identify assumptions and uncertainties, then verify important claims independently.

General template

```
Role: You are a [relevant role or area of expertise].

Context:
- Goal or background: [what I am working on]
- Environment: [language, version, framework, platform]
- Relevant material: [code, data, notes, or error message]
- Constraints: [requirements, restrictions, and what must not change]

Task: [one clear action, with success criteria]

Output:
- Format: [patch, steps, table, explanation, etc.]
- Include: [tests, assumptions, trade-offs, references, etc.]
- If information is missing, ask or state the assumption rather than inventing details.
```

Example: improve a vague request

Vague: “Make this function faster.”

Structured: “Review the Python function below. Explain its current time and space complexity, identify the main bottleneck, and propose the smallest change that improves performance without changing its behavior. Keep the public function signature unchanged. Show the revised function, explain the complexity, and provide a small test covering empty, typical, and duplicate-value inputs. If the available context is insufficient to establish behavior, list the assumption.”

3. Development and coding workflows

Smart code refactoring

Use this when improving readability or structure without changing behavior.

```
Role: Act as a careful software engineer reviewing a refactor.

Context:
- Language and version: [e.g., Python 3.12]
- Intended behavior: [describe it]
- Constraints: [public API, dependencies, style, compatibility]
- Code: [paste the relevant code]

Task: Suggest a focused refactor. Preserve behavior and avoid unrelated changes.

Output:
1. Summarize the current behavior and any assumptions.
2. Provide the revised code or a minimal patch.
3. Explain the trade-offs.
4. Analyze time and space complexity before and after, where meaningful.
5. Suggest tests that would detect behavior changes.
Do not claim a complexity improvement unless it follows from the code and input model.
```

Context-aware debugging

Provide the smallest useful reproduction, exact error text, expected behavior, and observed behavior. Remove secrets and unrelated code.

```
Role: Act as a methodical debugging partner.

Context:
- Environment and versions: [language, runtime, framework]
- Expected behavior: [what should happen]
- Actual behavior: [what happens]
- Exact error or traceback: [paste it]
- Minimal relevant code and input: [paste them]
- Recent changes or attempted fixes: [if known]

Task: Find the most likely cause and recommend a minimal fix.

Output:
1. Point to the specific line or condition implicated by the evidence.
2. Explain the cause and distinguish confirmed facts from hypotheses.
3. Show a minimal patch.
4. Give a command or test to verify the fix.
If the evidence is insufficient, tell me what additional observation would narrow it down.
```

Docstring and documentation generator

Generated documentation should describe actual behavior, not inferred intentions. Review parameters, return values, exceptions, examples, and edge cases against the implementation.

```
Role: Act as a technical writer familiar with [language or documentation standard].

Context:
- Documentation convention: [e.g., Google-style Python docstrings]
- Audience: [maintainers, API users, students]
- Code: [paste the relevant declaration and implementation]

Task: Write or improve documentation for this code without changing its behavior.

Output:
- Provide the docstring or documentation section only.
- Document parameters, return values, side effects, and exceptions that are evident from the code.
- Include a concise example only if it can be derived reliably.
- Mark unclear behavior as a question; do not invent guarantees.
```

4. Study and research workflows

Feynman study assistant

Use this workflow to test understanding, surface gaps, and build an explanation in your own words. An analogy is a starting point, not proof that the underlying concept is equivalent.

```
Role: Be a patient computer science tutor. Help me reason rather than simply giving me an answer.

Context:
- Topic: [algorithm, data structure, or concept]
- My current explanation or understanding: [explain it in your own words]
- Course level and known prerequisites: [describe them]

Task: Help me identify gaps and explain the topic using the Feynman technique.

Output:
1. Ask one focused question about my explanation.
2. After I respond, point out what is correct and what needs refinement.
3. Offer a plain-language explanation and a clearly labeled analogy.
4. Walk through a small concrete example.
5. Finish with a short recall question.
Do not skip reasoning steps or treat an analogy as a formal definition.
```

Research and lecture-note summarizer

Provide the source text or excerpts and request a summary grounded in that material. The model may miss nuance; check the original before citing or relying on a summary.

```
Role: Act as a careful technical reading assistant.

Context:
- Source type and topic: [paper, lecture notes, documentation]
- Intended use: [revision, comparison, background]
- Source text or excerpt: [paste text or permitted excerpts]

Task: Summarize the provided material without adding unsupported claims.

Output:
1. Main question or purpose.
2. Key ideas and terminology.
3. Methods, assumptions, or prerequisites.
4. Main results or conclusions stated in the source.
5. Limitations, open questions, or unclear passages.
6. A short list of concepts to review.
Distinguish the source's claims from your interpretation. If a detail is not in the supplied text, say so.
```

5. Productivity and exam preparation

Active recall

Turn study material into questions that require retrieval, not just rereading.

```
Using only the notes below, create [number] active-recall questions about [topic].
Mix definition, compare/contrast, explain-why, and worked-example questions.
Do not show the answers next to the questions. Put a concise answer key in a separate section.
Flag any question whose answer is not supported by the notes.

Notes:
[paste your notes]
```

Try answering before revealing the key. Correct misunderstandings using course materials, and revisit difficult questions later.

Feature breakdown planning

```
Role: Act as a software project planning partner.

Context:
- Feature goal: [user outcome]
- Existing system or codebase: [relevant details]
- Constraints and dependencies: [list them]

Task: Break the feature into small, verifiable implementation steps.

Output:
- Assumptions and questions that block planning
- Ordered tasks with a clear outcome for each
- Relevant tests or acceptance criteria
- Risks and dependencies
Keep the plan scoped to the feature; do not invent product requirements.
```

Standardized Git commit messages

```
Review the change summary below and propose one concise commit message.
Use this format: <type>(<optional scope>): <imperative summary>
Choose a type such as feat, fix, docs, refactor, test, or chore.
Do not claim changes that are not in the summary.
If the change contains unrelated work, suggest splitting it into separate commits.

Change summary:
[describe the staged changes]
```

Always inspect the staged diff yourself before committing. A generated message is a suggestion, not a substitute for reviewing what will be committed.

6. Quick reference

Before sending a prompt

- Is the goal specific?
- Have I included the relevant code, input, error, or source material?
- Have I stated constraints and what must remain unchanged?
- Have I specified the desired output?
- Have I removed secrets and information I should not share?

Choose a workflow

Need	Workflow
Improve structure without changing behavior	Smart code refactoring
Investigate a failure	Context-aware debugging
Describe existing code	Documentation generator
Learn a difficult concept	Feynman study assistant
Extract key points from provided material	Research summarizer
Prepare for an exam	Active recall
Plan an implementation	Feature breakdown
Summarize staged work	Git commit message

Ask for verifiable output

For code tasks, request tests or a verification command. For explanations, ask for a worked example. For summaries, ask the model to separate source claims from interpretation. For every task, ask it to state uncertainty instead of filling gaps

with invented details.

Responsible use

- Review and test generated code before running or shipping it.
- Do not paste credentials, private data, or code you are not permitted to disclose.
- Verify factual claims, citations, complexity analysis, and academic requirements.
- Follow your institution's and workplace's rules on AI assistance and attribution.
- Treat AI as an aid to your judgment, not a replacement for it.

License

This guide is available under the MIT License.

Contributing

Suggestions for clearer prompts, useful examples, and corrections are welcome. Read CONTRIBUTING.md before opening an issue or pull request.